

1. [30 points]

Your goal is to train a generative adversarial network (GAN) as a generative model for RGB microscopy image patches of size 128×128 pixels.

- (10 points) Describe an appropriate convolutional neural-net architecture for all GAN components. Describe each layer in detail. Describe kernel sizes clearly. Describes outputs clearly.

Rubric:

- (5 marks) should specify at least U-Net for generator. without skip connections, cut 3 marks.
- (5 marks) should specify at least Alex-Net / VGG-Net / ResNet for discriminator/classifier.

- (10 points) Mathematically formulate the optimization problem for GAN training involving the neural-net parameters θ , given training data $\{X_n\}_{n=1}^N$. as in lecture slides

Solution:

Mathematical Formulation: The training of a Generative Adversarial Network (GAN) is formulated as a two-player minimax game between a discriminator D and a generator G . Let θ_D and θ_G represent the neural network parameters for the discriminator and generator, respectively.

Given the training dataset $\{X_n\}_{n=1}^N$ where $X_n \sim p_{data}$, and a latent noise distribution $p_z(z)$ (typically a standard Gaussian), the optimization problem is defined as:

$$\min_{\theta_G} \max_{\theta_D} V(D, G)$$

where the objective function $V(D, G)$ is:

$$V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x; \theta_D)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z; \theta_G); \theta_D))]$$

Empirical Objective for Training Data: For a finite training set $\{X_n\}_{n=1}^N$ and a set of N sampled latent vectors $\{z_n\}_{n=1}^N$, the empirical optimization objective for the parameters is:

- **Discriminator Update:** Maximize the log-likelihood of correctly classifying real and generated patches:

$$\max_{\theta_D} \frac{1}{N} \sum_{n=1}^N [\log D(X_n; \theta_D) + \log(1 - D(G(z_n; \theta_G); \theta_D))]$$

- **Generator Update:** Minimize the probability that the discriminator identifies generated patches as fake:

$$\min_{\theta_G} \frac{1}{N} \sum_{n=1}^N \log(1 - D(G(z_n; \theta_G); \theta_D))$$

Rubric:

- **Mathematical Formulation (10 marks):**
 - Correct minimax objective function $V(D, G)$ (5 marks).
 - Proper use of log-expectations for both real and synthetic data (5 marks).
 - -3 marks for improper notation

- (10 points) For a fixed generator, mathematically derive the mathematical expression for the optimal discriminator function. At the end of a successful run for GAN training, specify the optimal discriminator function mathematically and specify the optimal generator mapping statistically.

Solution:

Derivation of the Optimal Discriminator $D_G^*(x)$: The objective function for the GAN is given by the minimax game:

$$V(G, D) = \mathbb{E}_{x \sim p_{data}(x)}[\log D(x)] + \mathbb{E}_{z \sim p_z(z)}[\log(1 - D(G(z)))]$$

To find the optimal discriminator for a fixed generator G , we rewrite the expectations as integrals:

$$V(G, D) = \int_x p_{data}(x) \log D(x) dx + \int_z p_z(z) \log(1 - D(G(z))) dz$$

Applying a change of variables $x = G(z)$, the second term becomes:

$$V(G, D) = \int_x [p_{data}(x) \log D(x) + p_g(x) \log(1 - D(x))] dx$$

To maximize this functional, we perform pointwise maximization of the integrand. For any x , we look for the value of $y = D(x)$ that maximizes:

$$f(y) = a \log(y) + b \log(1 - y)$$

where $a = p_{data}(x)$ and $b = p_g(x)$. Taking the derivative and setting it to zero:

$$\frac{df}{dy} = \frac{a}{y} - \frac{b}{1 - y} = 0 \implies a(1 - y) = by \implies y = \frac{a}{a + b}$$

Thus, the optimal discriminator function is:

$$D_G^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_g(x)}$$

Optimal State at the End of Training: At the end of a successful training run, the following conditions are met:

- **Optimal Discriminator Function:** Since the generator has perfectly modeled the data distribution ($p_g = p_{data}$), the discriminator cannot distinguish between samples:

$$D^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_{data}(x)} = 0.5$$

- **Optimal Generator Mapping:** The generator mapping $G(z)$ statistically transforms samples from the **standard normal distribution** $p_z(z)$ into the **data distribution** $p_{data}(x)$.

Rubric:

- **Generator Mapping (5 marks):** Maps standard normal to data distribution.
- **Discriminator Mapping (5 marks):** Outputs 0.5 for any input image.

2. [30 points]

Consider a dataset $\{z_n \in \mathbb{R}^D\}_{n=1}^N$ obtained as latent-space encodings z_n of an autoencoder learned from N images of brain MRI of infants between 0 months and 12 months of age. Consider that some of these infants exhibit normal brain development and some have a brain-development-related disorder. Consider that you want to cluster the dataset into two groups. You have found that the distribution of encodings z for each group cannot be modeled by a multivariate Gaussian.

- (10 points) Mathematically formulate an optimization problem for the clustering task.

Solution:

We are given latent representations $\{z_n \in \mathbb{R}^D\}_{n=1}^N$ and aim to cluster them into $K = 2$ groups. Since clusters are not Gaussian, we use kernel k -means.

Let $\phi(z)$ be an implicit feature map induced by a kernel $k(z_i, z_j) = \langle \phi(z_i), \phi(z_j) \rangle$.

We use the Gaussian RBF kernel:

$$k(z_i, z_j) = \exp\left(-\frac{\|z_i - z_j\|^2}{2\sigma^2}\right)$$

Define cluster assignments $c_n \in \{1, 2\}$ and clusters:

$$\mathcal{C}_k = \{n : c_n = k\}, \quad N_k = |\mathcal{C}_k|$$

The kernel k -means objective is:

$$\min_{\{c_n\}} \sum_{k=1}^2 \sum_{n \in \mathcal{C}_k} \|\phi(z_n) - \mu_k\|^2$$

where cluster means in feature space are:

$$\mu_k = \frac{1}{N_k} \sum_{i \in \mathcal{C}_k} \phi(z_i)$$

Expanding using kernel trick:

$$\|\phi(z_n) - \mu_k\|^2 = k(z_n, z_n) - \frac{2}{N_k} \sum_{i \in \mathcal{C}_k} k(z_n, z_i) + \frac{1}{N_k^2} \sum_{i, j \in \mathcal{C}_k} k(z_i, z_j)$$

Final optimization:

$$\min_{\{c_n\}} \sum_{k=1}^2 \sum_{n \in \mathcal{C}_k} \left[k(z_n, z_n) - \frac{2}{N_k} \sum_{i \in \mathcal{C}_k} k(z_n, z_i) + \frac{1}{N_k^2} \sum_{i, j \in \mathcal{C}_k} k(z_i, z_j) \right]$$

Rubric:

- Correct kernel k -means objective (5 pts)
- Proper definition of feature map and means (2 pts)
- Use of Gaussian RBF kernel (1 pts)
- Correct kernelized distance expansion (2 pts)

- (20 points) Derive an implementable algorithm for solving the clustering problem. Give the pseudo-code for the algorithm. You cannot assume your programming environment has the capacity for auto-differentiation.

Solution:

We use alternating optimization:

Key idea: Cluster means μ_k are never computed explicitly; distances are computed using kernel evaluations.

Distance Computation:

Distance of point z_n to cluster k :

$$d(n, k) = k(z_n, z_n) - \frac{2}{N_k} \sum_{i \in \mathcal{C}_k} k(z_n, z_i) + \frac{1}{N_k^2} \sum_{i, j \in \mathcal{C}_k} k(z_i, z_j)$$

Algorithm Steps:

- Initialize cluster labels c_n randomly.
- Compute kernel matrix $K_{ij} = k(z_i, z_j)$.
- Repeat until convergence:

- For each cluster k , compute:

$$S_k = \sum_{i, j \in \mathcal{C}_k} K_{ij}$$

- For each point n and cluster k , compute:

$$d(n, k) = K_{nn} - \frac{2}{N_k} \sum_{i \in \mathcal{C}_k} K_{ni} + \frac{1}{N_k^2} S_k$$

- Update label:

$$c_n = \arg \min_{k \in \{1, 2\}} d(n, k)$$

- Stop when labels do not change.

Pseudocode:

Input: Data $\{z_n\}$, kernel width σ

Compute $K[i, j] = \exp(-(\|z_i - z_j\|^2) / (2 * \sigma^2))$

Initialize labels $c[n]$ in $\{1, 2\}$

repeat:

 for k in $\{1, 2\}$:

```

Ck = {i such that c[i] == k}
Nk = size of Ck
Sk = sum over i,j in Ck of K[i,j]

for n = 1 to N:
  for k in {1, 2}:
    term1 = K[n,n]
    term2 = (2 / Nk) * sum over i in Ck of K[n,i]
    term3 = Sk / (Nk * Nk)
    d[k] = term1 - term2 + term3

    c[n] = k that minimizes d[k]

until convergence

Output: cluster labels c[n]

```

Rubric:

- Correct distance expression using kernel trick (5 pts)
- Clear alternating optimization explanation (3 pts)
- Proper label update rule (6 pts)
- Correct and complete pseudocode (6 pts)

3. [30 points]

Consider MRI imaging of a homogeneous piece of tissue, with standard semantics of the XYZ coordinate frame (as covered in class).

- (12 points) For the saturation-recovery process in MRI, plot a graph of the X component of the magnetization vector (magnitude and sign both) in a tissue as a function of time $t \in [0, \infty)$. Also specify the X values, as a function of time, using a mathematical expression.

Do the same for the Y component and the Z component, ensuring the time-axis values are consistent (i.e., in sync) across all three curves (to ensure this, you can also plot all three components on a single graph). Also specify the Y and Z values, as a function of time, using mathematical expressions. Clearly state all relevant rates and/or speeds underlying the processes.

Rubric:

(2 points) X component stays zero. see the equations in lecture slides.

(2 points) Y component stays zero. see the equations in lecture slides.

(4 points) Z component grows from zero to m_{eq} . see the equations in lecture slides.

(4 points) For Z, specify the growth as $m_{eq}(1 - \exp(-t/T_1))$.

- (18 points) Now consider that the tissue magnetization (non-zero) has reached the equilibrium state, and then a 90-degree pulse is applied. Considering a time scale where $t = 0$ is the time immediately after the pulse is switched off, plot a graph of the X component of the magnetization vector in a tissue as a function of time $t \in [0, \infty)$. Also specify the X values, as a function of time, using a mathematical expression. Do the same for the Y component and the Z component, ensuring the time-axis values are consistent (i.e., in sync) across all three curves (to ensure this, you can also plot all three components on a single graph). Also specify the Y and Z values, as a function of time, using mathematical expressions. Clearly state all relevant rates and/or speeds underlying the processes.

Solution:

X component shows sinusoidal oscillations that get dampened over time towards zero magnitude.

Y component shows sinusoidal oscillations that get dampened over time towards zero magnitude. Peaks of the X curve must match valleys of the Y curve.

Z component grows from zero to m_{eq} . see the equations in lecture slides.

For X and Y, specify frequency of oscillation, and rate of dampening as $\exp(-t/T_2)$.

For Z, specify the rate of growth as $m_{eq}(1 - \exp(-t/T_1))$.

Rubric:

- 6 marks each for the three directions, weightage is on the graph, if they are incorrect then no marks are awarded.

4. [30 points]

Consider N shapes (object boundaries) modeled as discretized simple closed curves in 2D images represented by pointsets $\{z_n\}_{n=1}^N$, with each pointset z_n comprising M points in $\mathbb{R}^2$. You want to compute a Bayesian estimate of a mean pointset μ , which includes a smoothness prior on the boundary represented by μ .

- (10 points) Mathematically formulate an optimization problem for the task.

Solution:

We are given N shapes, each represented as a set of M ordered points in $\mathbb{R}^2$. Our goal is to estimate a mean shape μ , while also ensuring that this mean shape is smooth, since it represents a boundary.

A convenient way to model this is using a Bayesian (MAP) formulation. This leads to an objective consisting of:

- a data term (how well the mean explains the observed shapes), and
- a prior term (enforcing smoothness of the mean shape).

Let $\mu \in \mathbb{R}^{M \times 2}$ denote the mean shape, where μ_m is the m -th point. Since the observed shapes may differ by translation, rotation, and scale, we introduce alignment parameters for each shape:

$$s_n > 0, \quad R_n \in \{R \in \mathbb{R}^{2 \times 2} \mid R^\top R = I, \det(R) = 1\}, \quad t_n \in \mathbb{R}^2.$$

Data term: We measure the discrepancy between the aligned shapes and the mean using a squared Frobenius norm:

$$E_{\text{data}} = \sum_{n=1}^N \|s_n z_n R_n + \mathbf{1} t_n^\top - \mu\|_F^2.$$

Smoothness prior: To encourage μ to form a smooth closed curve, we penalize differences between consecutive points:

$$E_{\text{prior}} = \lambda \sum_{m=1}^M \|\mu_m - \mu_{(m \bmod M)+1}\|_2^2,$$

where $\lambda > 0$ controls the strength of the smoothness.

Final objective:

$$\min_{\mu, \{s_n, R_n, t_n\}} \left[\sum_{n=1}^N \|s_n z_n R_n + \mathbf{1} t_n^\top - \mu\|_F^2 + \lambda \sum_{m=1}^M \|\mu_m - \mu_{(m \bmod M)+1}\|_2^2 \right]$$

Rubric:

- 3 points - Variables: Correctly identified μ and alignment parameters (s_n, R_n, t_n) , or handled implicitly with justification.
- 4 points - Likelihood Term: Proper formulation of a data fidelity term (e.g., Procrustes distance).
- 3 points - Prior Term: Smoothness penalty on adjacent points with wrap-around for closed curve.

- (20 points) Derive an implementable algorithm for solving the mean-estimation problem. Give the pseudo-code for the algorithm. You cannot assume your programming environment has the capacity for auto-differentiation.

Solution:

Directly optimizing all variables together is difficult, so we use an alternating minimization approach:

- Fix μ , update alignments
- Fix alignments, update μ

Preprocessing: To simplify the problem, we normalize each shape:

- subtract its mean (remove translation)
- scale to unit Frobenius norm

This removes the need to explicitly optimize s_n and t_n within the iterative loop. We enforce the same normalization on μ after every update.

Updating rotations: With μ fixed, each $R_n \in \{R \in \mathbb{R}^{2 \times 2} \mid R^\top R = I, \det(R) = 1\}$ is obtained by solving the Orthogonal Procrustes problem:

$$\min_{R_n} \|z_n R_n - \mu\|_F^2$$

Expanding the Frobenius norm, minimizing this distance is mathematically equivalent to maximizing the trace $\text{Tr}(R_n^\top z_n^\top \mu)$. Let the cross-covariance matrix be:

$$H = z_n^\top \mu$$

Compute the Singular Value Decomposition (SVD) of H :

$$H = U \Sigma V^\top$$

To maximize the trace, we set:

$$R_n = UV^\top$$

To ensure R_n is a valid rotation (and not a reflection), check the determinant. If $\det(R_n) < 0$, flip the sign of the last column of U and recompute $R_n = UV^\top$.

Updating the mean: With rotations fixed, we take the derivative of the total energy function w.r.t. μ . The gradient of the data fidelity term is $-2 \sum_{n=1}^N (z_n R_n - \mu)$, and the gradient of the smoothness prior is $2\lambda K \mu$. Thus, the total gradient is:

$$\nabla_\mu = 2N\mu - 2 \sum_{n=1}^N z_n R_n + 2\lambda K \mu$$

where $K \in \mathbb{R}^{M \times M}$ is defined entry-wise by

$$K_{ij} = \begin{cases} 2, & i = j, \\ -1, & j = i - 1 \text{ or } j = i + 1, \\ -1, & (i, j) = (1, M) \text{ or } (M, 1), \\ 0, & \text{otherwise.} \end{cases}$$

We update μ using gradient descent with learning rate α :

$$\mu \leftarrow \mu - \alpha \nabla_{\mu}$$

Projection step: To project the updated mean back into the normalized shape space:

$$\mu \leftarrow \mu - \text{mean}(\mu), \quad \mu \leftarrow \frac{\mu}{\|\mu\|_F}$$

Algorithm:

Rubric:

- 4 points - Translation & Scale: Correct preprocessing or valid updates for s_n, t_n .
- 6 points - Rotation: Correct Procrustes/Kabsch solution (including SVD and reflection handling) or equivalent valid method.
- 6 points - Mean Update: Correct derivation and incorporation of both the data fidelity gradient and the smoothness prior gradient.
- 4 points - Structure: Clear alternating minimization framework and proper normalization/projection of μ .

Algorithm 1 Alternating Minimization for Mean Shape Estimation

Require: Pointsets $\{z_1, \dots, z_N\}$, learning rate α , smoothness weight λ , max iterations T

Ensure: Optimized mean shape μ

▷ **Preprocess**

```
1: for  $n = 1$  to  $N$  do  
2:    $z_n \leftarrow z_n - \text{mean}(z_n)$   
3:    $z_n \leftarrow \frac{z_n}{\|z_n\|_F}$   
4: end for
```

▷ **Initialize**

```
5:  $\mu \leftarrow z_1$   
6: Construct Laplacian matrix  $K$ 
```

```
7: for  $t = 1$  to  $T$  do
```

▷ **Update rotations**

```
8:   for  $n = 1$  to  $N$  do  
9:      $H \leftarrow z_n^\top \mu$   
10:     $U, \Sigma, V^\top \leftarrow \text{SVD}(H)$   
11:     $R_n \leftarrow UV^\top$   
12:    if  $\det(R_n) < 0$  then  
13:      Flip the last column of  $U$   
14:       $R_n \leftarrow UV^\top$   
15:    end if  
16:  end for
```

▷ **Update mean**

```
17:   $\nabla_\mu \leftarrow 2N\mu - 2 \sum_{n=1}^N (z_n R_n) + 2\lambda K \mu$   
18:   $\mu \leftarrow \mu - \alpha \nabla_\mu$ 
```

▷ **Normalize (Projection)**

```
19:   $\mu \leftarrow \mu - \text{mean}(\mu)$   
20:   $\mu \leftarrow \frac{\mu}{\|\mu\|_F}$   
21: end for
```

```
22: return  $\mu$ 
```
